FastAPI and ML Integrated AI-Agent

An **Application Programming Interface (API)** is a software intermediary that allows two separate applications to communicate with each other. When you use an app on your phone to check the weather, the app sends a request to a weather bureau's API, which processes the request and returns the data to your screen. APIs are the backbone of modern web services, mobile app backends, and microservice architectures.

FastAPI is a modern, high-performance web framework for building APIs with Python, based entirely on standard Python type hints.

Why FastAPI?

- **Extremely Fast:** It performs on par with Node.js and Go, making it one of the fastest Python frameworks available. This speed comes from its underlying technologies: **Starlette** (for web routing) and **Pydantic** (for data validation).
- **Fast to Code:** By utilizing Python type hints, you get excellent editor support (autocompletion) and reduce the time spent debugging data types.
- **Automatic Documentation:** It automatically generates interactive API documentation (Swagger UI and ReDoc) based on your code, allowing you to test endpoints directly from your browser.
- **Built-in Async Support:** It natively supports asynchronous programming (`async def`), allowing it to handle concurrent requests efficiently.

Installation

You can install FastAPI and its standard dependencies (which includes the `uvicorn` server) using pip:

```
pip install "fastapi[standard]"
```

FastAPI Basics in Detail

HTTP Methods

HTTP methods indicate the desired action to be performed for a given resource. FastAPI supports all standard methods:

- **GET:** Read or retrieve data.
- **POST:** Create new data.
- **PUT:** Update existing data entirely.
- **PATCH:** Partially update existing data.
- **DELETE:** Remove data.

Request & Response Lifecycle

1. **Client Request:** A client (browser, mobile app) sends an HTTP request to an endpoint (e.g., `POST /items`).
2. **ASGI Server (Uvicorn):** Receives the request and passes it to the FastAPI application.
3. **Routing:** FastAPI routes the request to the matching function based on the path and method.
4. **Validation (Pydantic):** FastAPI extracts and validates parameters and body data. If the data is invalid, it instantly returns an automatic 422 Error to the client.
5. **Execution:** Your Python function runs its logic (often interacting with a database).
6. **Serialization:** The returned Python data (e.g., a dictionary or database model) is automatically converted to JSON.
7. **Response:** The server sends the HTTP response back to the client.

Request Parameters

FastAPI maps parts of the HTTP request to your function arguments based on how you define them:

- **Path Parameters:** Variables embedded in the URL path. Example: `/users/{user_id}`.
- **Query Parameters:** Key-value pairs attached to the end of the URL. Example: `/users?limit=10&sort=asc`.
- **Request Body:** JSON data sent by the client, typically used in POST/PUT requests. You define this using a Pydantic `BaseModel`.
- **Headers & Cookies:** Read directly from the HTTP headers using FastAPI's `Header()` and `Cookie()` functions.

Response Types

By default, FastAPI automatically converts your return values into standard JSON responses (`JSONResponse`). You can easily customize this:

- **Status Codes:** Return specific HTTP status codes (e.g., `201 Created` for POST requests).

- **Response Models:** Define exactly what data structure should be returned, ensuring you don't accidentally leak sensitive fields like passwords.
- **Custom Responses:** You can return raw HTML (`HTMLResponse`), plain text (`PlainTextResponse`), or file streams (`FileResponse`).

Simple FastAPI Setup

Here is a complete, simple application demonstrating path parameters, query parameters, and a request body.

Create a file named `main.py` :

```
from fastapi import FastAPI, status
from pydantic import BaseModel

# Initialize the application
app = FastAPI()

# Define a Pydantic model for the request body
class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

# GET method: Basic root endpoint
@app.get("/")
def read_root():
    return {"message": "Welcome to my FastAPI application!"}

# GET method: Path and Query parameters
@app.get("/items/{item_id}")
def read_item(item_id: int, q: str | None = None):
    # item_id is a path parameter, q is an optional query parameter
    return {"item_id": item_id, "query": q}

# POST method: Request body and specific status code
@app.post("/items/", status_code=status.HTTP_201_CREATED)
def create_item(item: Item):
    # 'item' is parsed from the JSON request body
    total_price = item.price
    if item.tax:
        total_price += item.tax

    return {
        "item_name": item.name,
        "total_price": total_price,
        "status": "Item created successfully"
    }
```

Run the API and Test It

To run the API locally, open your terminal in the same directory as `main.py` and run the development server:

```
fastapi dev main.py
```

(Alternatively, if you installed older versions, use: `uvicorn main:app --reload`)

How to test it:

1. **Test the Root:** Open your browser and go to `[http://127.0.0.1:8000](http://127.0.0.1:8000)`. You will see `{"message":"Welcome to my FastAPI application!"}`.
2. **Test Parameters:** Go to `[http://127.0.0.1:8000/items/5?q=apple](http://127.0.0.1:8000/items/5?q=apple)`. You will see the parsed path ID (5) and query string ("apple").
3. **Test the Interactive Docs (Swagger UI):** This is FastAPI's best feature. Go to `[http://127.0.0.1:8000/docs](http://127.0.0.1:8000/docs)`.

- You will see a beautifully generated UI listing all your routes.
- Click on the **POST /items/** route.
- Click the **Try it out** button.
- Modify the generated JSON body and click **Execute**. You will see the server's response right in the browser.

4. **Alternative Docs (ReDoc):** Go to `[http://127.0.0.1:8000/redoc](http://127.0.0.1:8000/redoc)` to see a different, more reading-focused style of automatic API documentation.

---

www.Vi4Academy.com